Développer un Pokédex avec une API REST



Durée : 8 séances de 4 heures, soit 32 heures

Technologies : Python, FastAPI, Pydantic, SQLite, requests et PokéAPI

1. Présentation du projet

1.1. Le contexte

Vous devez développer le serveur d'une application web permettant de consulter des informations sur les Pokémon : un Pokédex.

Nous allons nous concentrer uniquement sur la **première génération** de Pokémon, c'est-à-dire le premier ensemble de Pokémon jamais créé.

La **première génération** (souvent abrégée **Gen 1**) désigne les **151 premiers Pokémon**, introduits en 1996 au Japon avec les jeux **Pokémon Rouge et Vert** (puis

Rouge et Bleu en Occident, 1999). Cette génération est aussi appelée **Kanto**, du nom de la région où se déroulent ces jeux.

Des pages HTML représentant le Pokédex vous seront fournie. Ces pages afficheront les Pokémon et permettront de consulter les fiches déjà enregistrées et de lancer un scan sur les Pokémon non découverts. Vous n'avez pas à développer le HTML, le CSS ou le JavaScript : votre travail porte sur le backend Python et sur l'API REST utilisée par la page.

Votre application utilisera deux sources de données :

- **PokéAPI**, une API externe qui fournit les informations des Pokémon (https://pokeapi.co/).
- **SQLite**, une base de données locale dans laquelle vous enregistrerez les Pokémon que vous aurez récupérés.

1.2. Le principe de fonctionnement

Le Pokédex ne doit pas être rempli dès le départ.

Le but n'est pas de créer un script qui télécharge automatiquement les 151 Pokémon. Vous devez récupérer les données d'un Pokémon individuellement, uniquement lorsqu'un utilisateur souhaite le consulter et demande son scan.

Cette règle est importante : votre serveur ne doit pas surcharger inutilement PokéAPI en effectuant des centaines de requêtes dont les données ne sont pas encore nécessaires.

1.3. Livrable attendu

Vous devrez remettre le code source complet de l'API sur Github dans un répertoire **PUBLIC**.

Les pages HTML sont fournies : vous n'avez pas à développer le frontend ni à le versionner dans votre répertoire.

Rendez-vous dans la partie `Ressources` du module dans Moodle pour télécharger les fichiers.

2. Préparer le projet

Créez un dossier nommé `pokedex` .

Dans ce dossier, créez un fichier `main.py`.

```
pokedex/
└── main.py
```

Créez et activez un environnement virtuel, puis installez les dépendances :

```
python -m venv .venv
```

Sous Windows :

```
.venv\Scripts\activate
```

Sous macOS ou Linux :

```
source .venv/bin/activate
```

Installez FastAPI et requests :

```
pip install fastapi[standard] requests
```

⚠️ Pensez à ajouter un fichier .gitignore à la racine de votre projet pour ne pas partager des fichiers inutiles, vous pouvez ajouter dans le fichier :

```
.venv
```

Lorsque vous développez, vous installez plusieurs bibliothèques Python pour faire fonctionner votre application. Cependant, **ces bibliothèques ne sont pas automatiquement incluses lorsque vous partagez votre code avec quelqu'un d'autre**.

Imaginez que vous envoyiez votre projet à un camarade ou que vous le déposiez sur GitHub. Cette personne récupère vos fichiers Python, mais elle ne possède

pas nécessairement les bibliothèques que vous avez installées sur votre ordinateur.

Le fichier `requirements.txt` sert à résoudre ce problème : il liste les dépendances nécessaires pour que quelqu'un d'autre puisse installer l'environnement requis et exécuter votre projet.

```
pip freeze > requirements.txt
```

Cette commande enregistre les versions des paquets présents dans l'environnement courant.

⚠️ Pensez à remettre le fichier à jour lors de l'ajout d'une nouvelle dépendances (en Python, ce n'est pas automatique).

Maintenant toute personne récupérant le projet peut installer toutes les dépendances nécessaires au projet avec cette commande :

```
pip install -r requirements.txt
```

3. Créer l'application FastAPI

Dans `main.py`, créez une application FastAPI et une première route `GET /health`.

Cette route doit renvoyer :

```
{
  "status": "ok"
}
```

Lancez le serveur :

```
fastapi dev main.py
```

Testez la route `/health` depuis la documentation interactive (`http://127.0.0.1:8000/docs`) ou toutes autres méthodes (Postman, navigateur,...)

4. Consulter les informations d'un Pokémon

PokéAPI fournit les données d'un Pokémon à partir de son numéro.

Par exemple :

`https://pokeapi.co/api/v2/pokemon/25`

Ouvrez cette adresse dans un navigateur et observez les données retournées.

Dans `main.py`, créez une route :

`GET /pokemons/{number}`

Cette route devra dans un premier temps

1. recevoir un numéro de Pokémon ;
2. construire l'URL correspondante de PokéAPI ;
3. appeler PokéAPI avec `requests.get()` ;
4. vérifier si la requête a échoué ;
5. extraire les informations utiles ;
6. renvoyer un dictionnaire JSON.

Vous pouvez commencer avec les champs suivants :

- `number`
- `name`
- `height`
- `weight`

Tests à effectuer :

- Consulter le numéro 1.
- Consulter le numéro 25.
- Consulter un numéro inexistant.

Votre serveur fonctionne, mais il contacte encore PokéAPI à chaque consultation. Il ne dispose pas encore d'une base de données et toutes les responsabilités sont regroupées dans `main.py`.

Cette première version est volontairement simple. Vous allez la faire évoluer sans repartir de zéro.

5. Créer un modèle de données avec Pydantic

Une fiche Pokémon doit contenir :

Champ	Type attendu dans l'API	Description
`number`	Entier	Numéro national du Pokémon
`name`	Chaîne	Nom utilisé par PokéAPI
`name_fr`	Chaîne	Nom français du Pokémon
`types`	Liste de chaînes	Types du Pokémon traduits en français
`image_url`	Chaîne	URL de l'image par défaut du Pokémon
`height`	Entier	Taille du Pokémon en centimètres
`weight`	Entier	Poids du Pokémon en grammes
`evolves_from_number`	Entier ou valeur nulle	Numéro du Pokémon dont il évolue, si applicable

Le numéro Pokédex doit être unique dans votre base.

Pour le moment, ajoutez un modèle Pydantic dans `main.py` qui correspond aux données d'un Pokémon.

Il doit décrire les données que votre API souhaite renvoyer.

Vous pourrez utiliser ce modèle plus tard comme `response_model` sur votre route FastAPI.

```
@app.get("/pokemons/{number}", response_model=Pokemon)
def get_pokemon(number: int):
    return {
        "number": 25,
        "name": "pikachu"
    }
```

Ici, `response_model=Pokemon` indique à FastAPI que la réponse doit respecter le modèle `Pokemon`.

Pydantic permet de formaliser les données attendues en entrée ou en sortie. Il ne remplace pas la base de données : son rôle est différent.

- Pydantic décrit et valide les données échangées par l'API.
- SQLite conserve les données.
- FastAPI utilise les modèles Pydantic pour documenter et valider les échanges en entrée (comme vu en TD) ou en sortie.

6. Créer la base SQLite

Créez un fichier `create_database.py` et ajoutez-y ce code :

```
import sqlite3

connection = sqlite3.connect("pokedex.db")
cursor = connection.cursor()

cursor.execute("""
    CREATE TABLE IF NOT EXISTS pokemon (
        number INTEGER PRIMARY KEY,
        name TEXT NOT NULL,
        name_fr TEXT NOT NULL,
        types TEXT NOT NULL,
        image_url TEXT NOT NULL,
        height INTEGER NOT NULL,
        weight INTEGER NOT NULL,
        evolves_from_number INTEGER
    )
""")

connection.commit()
connection.close()
```

```
print("Base de données créée avec succès.")
```

L'initialisation de la base doit être effectuée une seule fois avec le script suivant :

```
python create_database.py
```

Votre base de données `pokedex` est maintenant créée ainsi que la table `pokemon`.

Si vous voulez lire le contenu d'un fichier SQLite `.db` , il est possible d'utiliser la librairie `SQLite3 Editor` dans Visual Studio Code :

https://marketplace.visualstudio.com/items?itemName=qwtel.sqlite-viewer

7. Créer le dossier des modèles

Votre modèle Pydantic n'a pas besoin de rester dans `main.py`.

Créez cette structure :

```
pokedex/
├── main.py
└── schemas/
    └── pokemon.py
```

Déplacez la définition du modèle Pokémon dans `schemas/pokemon.py`.

Le fichier doit contenir uniquement les modèles relatifs aux données échangées par l'application.

Dans `main.py`, supprimez la définition locale du modèle et importez-le :

```
from schemas.pokemon import Pokemon
```

Adaptez le nom `Pokemon` si vous avez choisi un autre nom pour votre classe.

Pourquoi cet import fonctionne-t-il ?

- `schemas` est le dossier du module ;
- `pokemon` correspond au fichier `pokemon.py` ;

- `Pokemon` est la classe définie dans ce fichier.

Python charge le module et rend la classe disponible dans `main.py`.

Vous n'avez pas besoin de recopier la classe dans chaque fichier, un import suffira désormais pour l'utiliser.

Pensez à supprimer les imports devenus inutiles dans `main.py` notamment Pydantic qui sera importé uniquement dans les `schemas`.

8. Créer le module de base de données

La connexion et l'initialisation de SQLite ne doivent pas être répétées dans chaque route.

Créez :

```
pokedex/
├── main.py
├── schemas/
│   └── pokemon.py
└── db/
    └── database.py
```

Placez dans `db/database.py` le code responsable :

- d'ouvrir une connexion SQLite ;
- de fournir une manière cohérente d'obtenir une connexion.

Dans `main.py`, importez les fonctions nécessaires depuis `db.database`.

Par exemple, le principe d'import est :

```
from db.database import get_connection
```

`get_connection` est ici un exemple de nom : utilisez celui de la fonction que vous avez réellement écrite.

9. Créer un repository

Les requêtes SQL sont différentes des requêtes HTTP. Elles méritent donc leur propre module.

Créez :

```
pokedex/
├── main.py
├── schemas/
│   └── pokemon.py
├── db/
│   └── database.py
└── repositories/
    └── pokemon_repository.py
```

Dans `repositories/pokemon_repository.py`, vous placerez les fonctions qui effectuent les opérations SQL sur les Pokémon.

Le repository reçoit des paramètres Python et renvoie les données trouvées ou le résultat de l'opération. Il ne doit pas construire de réponse HTTP FastAPI.

Dans `main.py`, vous pourrez importer les fonctions du repository nécessaires à vos premières routes.

Exemple de principe d'import

```
from repositories.pokemon_repository import get_pokemon_by_number
```

Là encore, adaptez le nom à votre implémentation.

10. Créer la route de scan

Ajoutez la route :

`GET /pokemons/{number}/scan`

Cette route doit demander l'importation d'un seul Pokémon.

Le numéro est transmis dans l'URL. Le serveur doit récupérer les informations correspondantes depuis PokéAPI, puis les enregistrer dans SQLite.

Le scan ne doit jamais déclencher l'importation de toute une liste.

Les appels à PokéAPI sont actuellement écrits dans le code de votre application.

Créez un module dédié :

```
pokedex/
└── clients/
    └── pokeapi.py
```

Dans `clients/pokeapi.py`, placez la fonction qui :

1. construit l'URL correspondant au numéro demandé ;
2. effectue la requête HTTP avec `requests` ;
3. détecte les erreurs HTTP ;
4. renvoie les données JSON.

Dans le fichier qui en a besoin, importez cette fonction. Par exemple :

```
from clients.pokeapi import get_pokemon_from_api
```

Le nom de la fonction doit correspondre à celui que vous avez défini.

Pourquoi créer ce module ?

Le client externe ne doit pas connaître les routes FastAPI ni les requêtes SQL.

Son rôle est uniquement de communiquer avec PokéAPI et de fournir les données obtenues.

Si vous devez modifier l'adresse ou la manière d'appeler PokéAPI, vous pourrez le faire dans ce module sans réécrire toutes les routes.

11. Normalisation des données

PokéAPI renvoie une structure JSON assez riche. Votre application n'a pas besoin de conserver toutes les données.

Documentation officielle : https://pokeapi.co/docs/v2

PokéAPI expose plusieurs ressources. Vous devrez apprendre à effectuer plusieurs requêtes HTTP pour obtenir une fiche complète.

11.1. Récupérer les informations principales

Pour récupérer les données d'un Pokémon, utilisez :

`https://pokeapi.co/api/v2/pokemon/{number}`

Par exemple, pour le numéro 25 :

`https://pokeapi.co/api/v2/pokemon/25`

Cette réponse contient notamment :

- `id` : le numéro du Pokémon
- `name` : son nom dans PokéAPI
- `types` : ses types, avec leurs identifiants
- `sprites.front_default` : l'URL de son image par défaut
- `height` : sa taille (attention aux unités)
- `weight` : son poids (attention aux unités)

11.2. Récupérer le nom français

La ressource principale d'un Pokémon ne fournit pas directement tous les noms localisés. Vous devez donc consulter sa ressource « espèce ».

Utilisez :

`https://pokeapi.co/api/v2/pokemon-species/{number}`

Dans la réponse, recherchez le tableau `names`. Chaque entrée contient un nom et une langue.

Vous devez sélectionner l'entrée dont :

`language.name == "fr"`

Le champ `name` de cette entrée correspond au nom français.

Vous devez prévoir le cas où aucune traduction française n'est disponible : ne faites pas planter toute l'application. Définissez un comportement de secours cohérent, par exemple utiliser le nom original.

La réponse principale de PokéAPI fournit les types avec des noms comme `electric`, `fire` ou `water`.

Pour obtenir leurs traductions, consultez la ressource du type correspondant :

`https://pokeapi.co/api/v2/type/{type_name}`

Par exemple :

`https://pokeapi.co/api/v2/type/electric`

Dans le tableau `names`, sélectionnez de nouveau l'entrée dont `language.name` vaut `"fr"`.

Votre backend doit donc :

1. Récupérer les types du Pokémon.
2. Lire le nom technique de chaque type.
3. Interroger la ressource de ce type pour obtenir sa traduction française.
4. Construire une liste contenant les noms français.
5. Enregistrer cette liste avec le Pokémon.

⚠️ SQLite ne possède pas de type natif correspondant à une liste de chaînes. Vous devez donc convertir la liste Python en texte JSON au moment de l'enregistrement.

```
import json

types = ["electric", "fire"]

# Encodage pour le stockage dans SQLite
types_json = json.dumps(types, ensure_ascii=False)
```

11.4. Déterminer de quel Pokémon il évolue

La ressource « espèce » contient également le champ `evolves_from_species`.

Ce champ désigne l'espèce dont le Pokémon actuel est issu directement dans la chaîne d'évolution.

Par exemple :

- Raichu évolue à partir de Pikachu.
- Bulbizarre n'a pas d'évolution précédente.

Pour récupérer cette information :

1. Consultez `https://pokeapi.co/api/v2/pokemon-species/{number}`.
2. Lisez `evolves_from_species`.
3. Si la valeur est `null`, le Pokémon n'a pas d'évolution précédente.
4. Déterminez son numéro à partir de l'identifiant présent à la fin de l'URL fournie par PokéAPI.

Il faut bien distinguer **le Pokémon dont le Pokémon actuel évolue** de son éventuelle évolution future. Ce projet ne demande que l'évolution précédente immédiate.

Vous n'avez pas à télécharger automatiquement toutes les autres espèces de la chaîne d'évolution.

> ⚠️ **Attention : cas particulier des pré-évolutions ajoutées après la Gen 1**
>
> Le champ `evolves_from_species` renvoie l'évolution **immédiate actuelle**, telle que définie par PokéAPI d'après les données les plus récentes du Pokédex national. Or, certaines pré-évolutions ont été **introduites dans des générations ultérieures** pour des Pokémon de la première génération, afin d'enrichir les chaînes d'évolution rétroactivement.
>
> | Pokémon de 1re génération | Pré-évolution ajoutée après |
> |---|---|
> | Pikachu (#025) | Pichu (#172) |
> | Mélofée (#035) | Mélo (#173) |
> | Rondoudou (#039) | Toudoudou (#174) |
> | Lippoutou (#124) | Lippouti (#238) |
> | Élektek (#125) | Élekid (#239) |
> | Magmar (#126) | Magby (#240) |
> | Kicklee / Tygnon (#106 / #107) | Debugant (#236) |
>
> **Règle à appliquer** : après avoir récupéré le numéro de l'espèce parent via `evolves_from_species`, **vérifiez que ce numéro est compris entre 1 et 151**.
>
> - Si le numéro est **≤ 151** : le Pokémon a bien une évolution précédente appartenant à la Gen 1 → conservez cette valeur.

> - Si le numéro est **> 151** : cette pré-évolution **n'existe pas en Gen 1** → traitez le Pokémon comme **n'ayant pas d'évolution précédente** (valeur `null` équivalente).
>
> Cette vérification est indispensable pour rester cohérent avec le périmètre du projet : les seules relations d'évolution retenues sont celles **internes aux 151 Pokémon de la première génération**.

12. Créer le service métier

Le scan devra respecter plusieurs règles :

- si le Pokémon est déjà enregistré, ne pas créer de doublon
- s'il est absent, récupérer les données externes
- convertir les données dans le format de votre application (normalisation de la données expliquée dans la partie précédente)
- enregistrer la fiche
- renvoyer les données enregistrées (en utilisant le modèle de réponse de la partie 5) ou une erreur `404 Not Found` si le Pokémon existe déjà

Ces règles ne doivent pas toutes être placées dans la route HTTP.

Créez :

```
pokedex/
└── services/
    └── pokemon_service.py
```

Dans `services/pokemon_service.py`, placez la logique métier du scan.

Le service doit utiliser le client PokéAPI pour récupérer les données et le repository pour accéder à SQLite.

Le trajet devient :

```
GET /pokemons/{number}/scan
              |
              v
        Route FastAPI
```

```
              |
              v
        Service métier
           /      \
          v        v
    Client API   Repository
          |        |
          v        v
       PokéAPI    SQLite
```

En résumé, l'architecture du projet sera la suivante :

Élément	Fonction principale
main.py	Créer l'application FastAPI et lister les routes.
schemas/	Définir et valider la structure des données entrantes et sortantes avec Pydantic.
services/	Appliquer les règles métier et coordonner les opérations entre les autres modules.
repositories/	Effectuer les opérations de lecture, d'écriture, de modification et de suppression en base de données.
clients/	Communiquer avec les services externes et récupérer leurs données.
db/	Gérer l'accès à la base de données et les connexions.

12.5. Optimisation de la traduction des types

Pour éviter d'interroger PokéAPI à chaque fois qu'un type doit être traduit, vous allez mettre en place un système de cache persistant dans SQLite.

L'objectif est de ne récupérer la traduction française d'un type qu'une seule fois, puis de la réutiliser lors des scans suivants, y compris après le redémarrage du serveur.

Un type doit contenir :

Champ	Type attendu dans l'API	Description
`id`	Entier	Identifiant du type de PokéAPI
`name`	Chaîne	Nom utilisé par PokéAPI
`name_fr`	Chaîne	Nom français du type

Mettez à jour le fichier `create_database.py` avec le code suivant pour créer la table correspondante et executez le script comme précédemment :

```
CREATE TABLE IF NOT EXISTS pokemon_types (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    name_fr TEXT NOT NULL
);
```

Fonctionnement :

Lors du scan d'un Pokémon, vérifier si chaque type est déjà présent dans cette table.

- Si le type est enregistré, utiliser directement sa traduction française.
- Si le type est absent, interroger PokéAPI, récupérer sa traduction française, puis l'enregistrer dans SQLite.

Veiller à respecter l'architecture du projet déjà mise en place (`schemas`, `repositories`, `clients`, `services`) et à répartir les responsabilités entre les différents modules. Le modèle devra être réutilisé de manière cohérente dans l'ensemble du projet.

13. Modifier la route d'un Pokémon

Modifiez `GET /pokemons/{number}`.

Désormais, la route doit :

1. rechercher le Pokémon dans SQLite (en pensant à utiliser le repository créé précédemment).
2. renvoyer la fiche si elle existe (en utilisant le modèle de réponse)
3. renvoyer `404 Not Found` si elle n'existe pas.

Attention : la route ne doit plus contacter PokéAPI pour une simple consultation.

Cette séparation est indispensable au fonctionnement du projet : un Pokémon non enregistré doit rester inconnu jusqu'à ce qu'un utilisateur demande son scan.

Tests à effectuer

- Consulter un Pokémon présent dans SQLite.
- Consulter un Pokémon absent.
- Vérifier que la consultation ne provoque aucun appel à PokéAPI.

14. Ajouter une route de liste

Créez la route :

`GET /pokemons`

Cette route doit renvoyer uniquement les Pokémon présents dans SQLite.

Elle ne doit pas contacter PokéAPI pour compléter automatiquement la liste.

15. Ajouter la sécurité

15.1. Comprendre l'authentification Bearer

Certaines opérations de votre API doivent être protégées.

Pour ce TP, la route de scan sera protégée par un jeton Bearer.

Un client autorisé transmet un en-tête HTTP de la forme :

```
Authorization: Bearer ENSEA2026
```

Le serveur doit vérifier le jeton avant d'autoriser le scan.

Un jeton absent ou invalide doit être refusé avec une réponse HTTP `401 Unauthorized`.

⚠️ Un jeton fixe suffit pour cet exercice, mais il ne représente pas une solution de sécurité complète pour une application publique.

15.2. Créer le module de sécurité

Ajoutez :

```
pokedex/
└── core/
    └── security.py
```

Dans `core/security.py`, créez une dépendance FastAPI qui vérifie le jeton Bearer.

La route de scan utilisera cette dépendance. La vérification ne doit pas être recopiée dans chaque fonction. La dépendance peut-être écrite de la manière suivante :

```
@app.get("/",dependencies=[Depends(verify_token)])
```

Là encore, adaptez le nom de la fonction à votre implémentation.

15.3. Protéger la route scan

Appliquez la dépendance de sécurité à `GET /pokemons/{number}/scan`.

Testez :

1. un scan sans jeton ;
2. un scan avec un jeton incorrect ;
3. un scan avec un jeton valide.

Seul le troisième cas doit permettre le scan.

16. Préparer l'intégration avec les pages HTML

Objectif : vérifier que votre API respecte le contrat attendu par les pages fournies.

16.1. Comprendre le rôle de la page

La page HTML `pokedex.html` représente le Pokédex et la page `pokemon_detail.html` représente la fiche d'un Pokémon. Elles contiennent déjà le code permettant d'envoyer des requêtes à votre API (veillez à ce que les 2 pages soient dans un même répertoire de votre ordinateur afin de fonctionner correctement).

Vous n'avez pas à développer cette interface.

En revanche, vous devez vous assurer que votre API renvoie les données et les codes HTTP attendus.

16.2. Communiquer entre différentes adresses

Dans notre projet, le Pokédex est composé de deux parties : la page HTML qui affiche les Pokémon et une API FastAPI qui fournit les données.

Ces deux parties peuvent être accessibles à des adresses différentes. Par exemple, la page HTML peut être ouverte à l'adresse `http://127.0.0.1:5500`, tandis que l'API est accessible à l'adresse `http://127.0.0.1:8000`.

Lorsqu'une page web cherche à récupérer des données auprès d'un serveur situé à une autre origine, le navigateur applique des règles de sécurité. Il peut bloquer la communication si le serveur n'autorise pas explicitement cette origine.

CORS (*Cross-Origin Resource Sharing*) est un mécanisme de sécurité qui permet à un serveur d'indiquer aux navigateurs quelles origines sont autorisées à accéder à ses ressources.

Pour permettre à notre page HTML de communiquer avec l'API FastAPI, nous devons donc configurer le CORS côté serveur.

Code à ajouter dans `main.py` :

```
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Dans cette configuration :

- `allow_origins=["*"]` autorise les requêtes provenant de toutes les origines.
- `allow_methods=["*"]` autorise toutes les méthodes HTTP, comme `GET` et `POST`.
- `allow_headers=["*"]` autorise tous les en-têtes HTTP.

Cette configuration convient à notre TP pour simplifier les échanges entre la page HTML et l'API. Dans une application réelle, il est préférable de limiter les origines autorisées aux adresses de confiance.

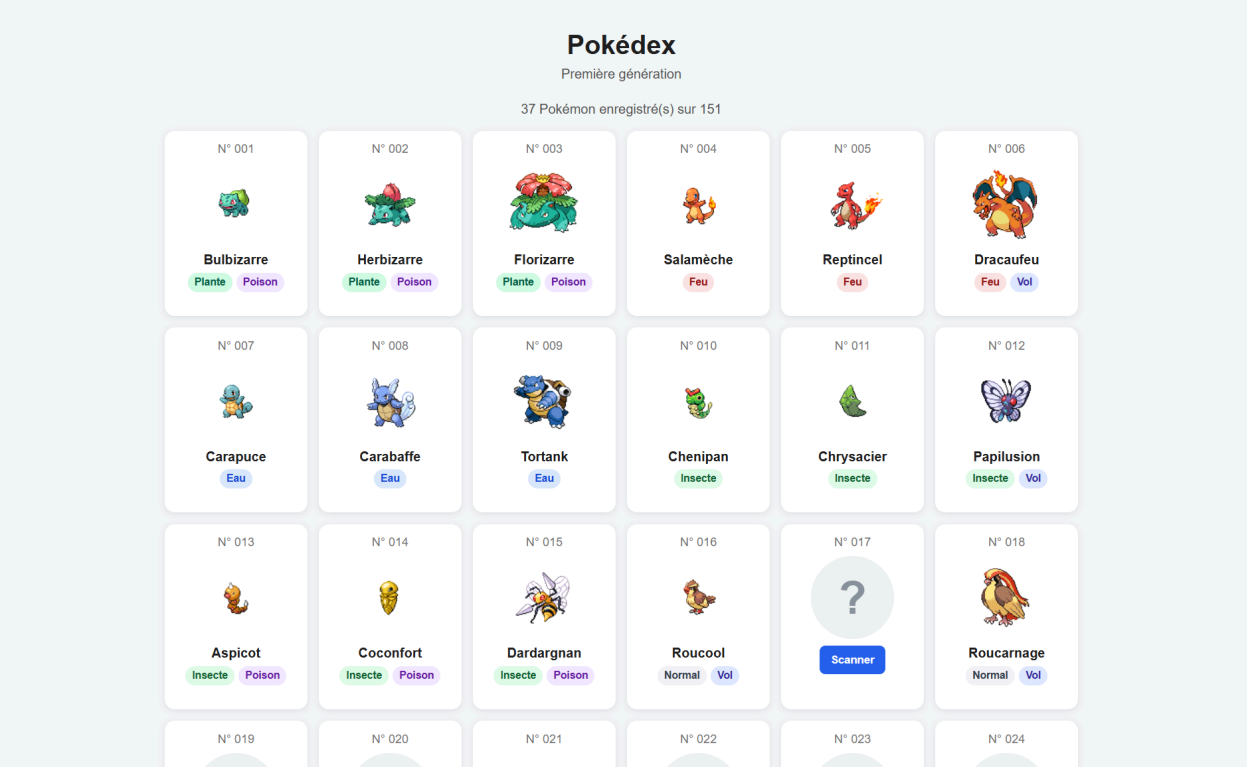
16.3. Vérifier l'affichage du Pokédex

La liste des Pokémon présent dans la base SQLite doivent être retournés pour être affichés par la page web `pokedex.html`.

`GET /pokemons`

L'API doit renvoyer les fiches au format JSON.

Les Pokémon manquants seront affichés avec un ? et un bouton de scan.



16.4. Vérifier le scénario de scan

Au clic sur « Scanner », la page appellera :

`GET /pokemons/{number}/scan`

Votre serveur devra récupérer un seul Pokémon depuis PokéAPI et l'enregistrer.

Le Pokémon devra être visible au rechargement de la page.

Vérifiez que le scan fonctionne également pour un numéro inexistant ou lorsque PokéAPI est indisponible.

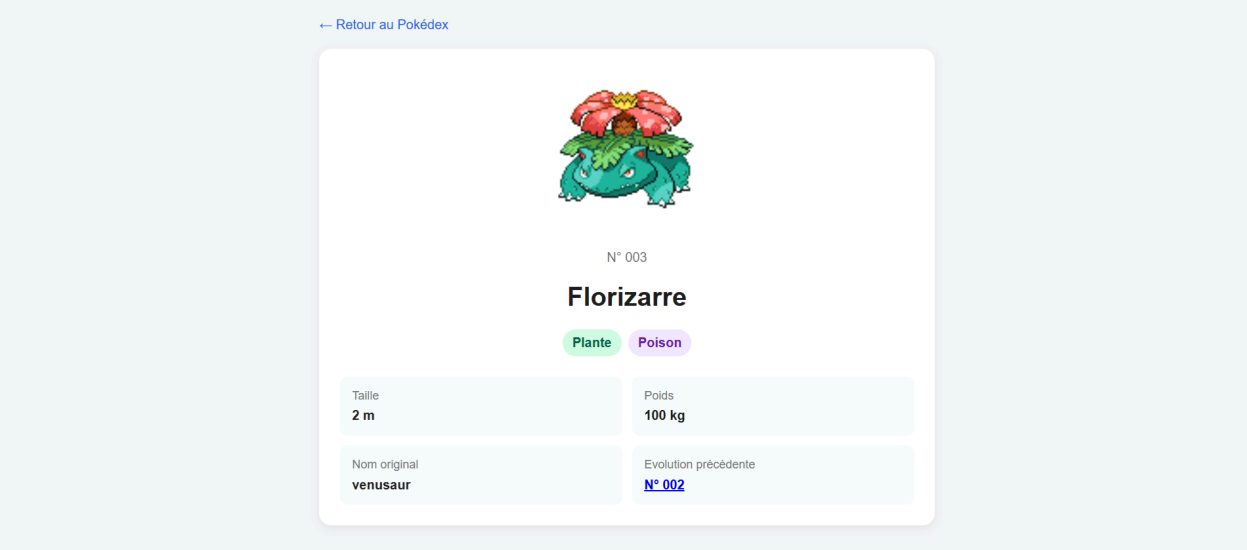
16.5. Vérifier le scénario de consultation

Pour un Pokémon enregistré, la page `pokemon_detail.html` doit pouvoir demander :

`GET /pokemons/{number}`

L'API doit renvoyer sa fiche au format JSON.

Pour un Pokémon absent, l'API doit renvoyer `404 Not Found`.



17. Finalisation et validation du projet

17.1. Vérification du fonctionnement

Avant de rendre votre projet, vérifiez que les fonctionnalités suivantes sont opérationnelles :

- L'API démarre correctement et la route `GET /health` renvoie un statut valide.
- La route `GET /pokemons` renvoie uniquement les Pokémon enregistrés dans la base SQLite.
- La route `GET /pokemons/{number}` renvoie la fiche d'un Pokémon enregistré ou une erreur `404 Not Found` s'il est absent.
- La route `GET /pokemons/{number}/scan` permet de récupérer et d'enregistrer un Pokémon depuis PokéAPI.
- Un Pokémon déjà enregistré n'est pas ajouté une seconde fois.
- Les informations enregistrées sont correctement normalisées : nom français, types traduits, image, taille, poids et évolution précédente.

- Le cache des types permet de réutiliser les traductions déjà enregistrées.
- La route de scan refuse les requêtes sans jeton Bearer valide, avec une réponse `401 Unauthorized`.
- L'API fonctionne avec les pages HTML fournies et le visuel correspond aux captures d'écran fournies plus haut.
- Les erreurs liées à un numéro de Pokémon inexistant ou à une indisponibilité de PokéAPI sont gérées proprement.

17.2. Qualité et organisation du code

Votre projet doit respecter les principes d'organisation étudiés pendant le TP :

- les routes FastAPI gèrent les requêtes HTTP et les réponses ;
- les modèles Pydantic décrivent les données échangées ;
- les repositories regroupent les opérations sur la base de données ;
- les clients gèrent les communications avec les API externes ;
- les services regroupent la logique métier ;
- les modules de sécurité centralisent la vérification du jeton Bearer.

Veillez également à supprimer les fichiers inutiles, les imports inutiles et à ne pas versionner votre environnement virtuel.

17.3. Bilan du projet

À l'issue de ce TP, vous devez être capables de développer une API REST en Python, de communiquer avec une API externe, de stocker et de consulter des données dans une base SQLite, et d'organiser une application en modules aux responsabilités distinctes.

Vous aurez également mis en pratique la validation des données, la gestion des erreurs HTTP, l'authentification par jeton Bearer et l'intégration d'un backend avec une interface web existante.